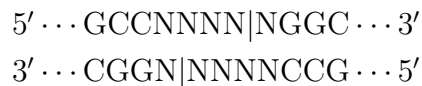


Numerical Optimization and Maximum Likelihood

As I mentioned in a previous assignment, there just aren't that many data fitting problems that you can do "by hand." Most of them rely on numerical optimization, so in this exercise, you are going to fit a distribution to some real data, setting the problem up using maximum likelihood and then solving it via numerically minimizing the negative log likelihood.

The first thing you will consider is DNA fragment sizes. There are enzymes called *restriction enzymes*, which are enzymes that cut DNA at specific sequences. The particular enzyme we will consider for this problem is called BglI, and its cut site looks like



where the vertical line indicates where the cut is made, and "N" represents any nucleotide (A,C,G, or T). BglI is a so-called "sticky-end" cutter because it leaves little overhangs in the two cut strands (the vertical lines aren't in the same place in the two strands). Genomes have many different restriction sites (each having a different recognition sequence), and usually have many copies of a given enzyme's cut site as well. Random cutting (where we just break up a genome at a given restriction site, yielding a bunch of different length fragments) is still used today when certain kinds of sequencing are done — particularly *shotgun sequencing*, which is the primary way we obtain microbiome sequences.

In this problem, you will start by doing some analysis and model fitting to the distribution of DNA sequence lengths produced by cutting the *E. coli* genome using BglI. You'll finish by fitting the same basic model to a completely different data set!

- (a) Start by reading in the contents of `BglI-fragments.fasta`. This file contains all the DNA sequence fragments produced by cutting the *E. coli* genome at every BglI cut site. Fragments are stored as one per line. You should compute the length of each fragment and store it in a list or array.
- (b) To plot the fragment length distribution, we will do a bit of binning. The reason for this is that there are lots of sequences that differ by only a base pair, so if you plot the raw count data (counts vs. length) it will be really noisy. I've provided a python function `bin_data()` for you that will create binned counts — binning essentially treats nearby samples (sequences close in length) as one entity and adds their counts together. Run this on your count data, and plot (just a dot plot is fine) the resulting empirical probability distribution of fragment lengths, with density on the y -axis and bin centers (length values) on the x -axis. You will probably want to experiment a bit with `nbins` to get a nice looking plot. (Too few bins and you won't see much detail, too many and your distribution will look really noisy.)
- (c) If we assume the sites are randomly distributed in the genome the simplest model for the length data is an exponential distribution, in which the probability of seeing a fragment of length l is

$$P(l|\mu) = \frac{1}{\mu} e^{-l/\mu}$$

where μ is a parameter we determine from the data. Write down the likelihood function for observing a set of independent lengths $\{l_i\}$ each distributed exponentially. Compute the negative log likelihood and minimize it to find the maximum likelihood estimate for μ (Hint: This calculation is pretty similar to what I did with the Poisson distribution in class.)

- (d) Now plot the length data again, with your fitted exponential distribution on top of it. Evaluate the exponential distribution at many different points to get a smooth curve (`numpy.linspace` will be really helpful for this). Does it look like a good fit?

You probably got a pretty good match to the length data with an exponential distribution, but other models are possible. One weird thing about the exponential distribution is that there is a nonzero probability for $l = 0$, which makes absolutely no sense in this context. For this reason, some people prefer a gamma distribution over an exponential one for problems like this, in which you will never observe any zeros in your dataset. The gamma probability density is

$$P(l|\alpha, \beta) = \frac{\beta^\alpha}{\Gamma(\alpha)} l^{\alpha-1} e^{-\beta l}$$

Here, $\Gamma(\alpha)$ is the Gamma function. We have to fit the two parameter gamma model numerically.

- (e) We will start by setting up the model. Write down the negative log likelihood for a set of independent length observations $\{l_i\}$ which are each gamma distributed.
- (f) We are going to do gradient descent, so we need a gradient. Compute the gradient of the negative log likelihood. (Hint: This will be a two-component vector since you need a derivative with respect to α and one with respect to β). You might be worried because you have a term that looks like

$$\frac{\Gamma'(\alpha)}{\Gamma(\alpha)}$$

and you don't know what to do with it. Fear not, because `scipy.special.digamma` is exactly this function! So when you need to evaluate it, you will be able to.

- (g) Before moving on, there is a trick we can use to make the fitting easier. Solve the $\partial l / \partial \beta$ equation for $\hat{\beta}$ and plug this into the $\partial l / \partial \alpha$ equation. This reduces the minimization to a one-dimensional problem in which all we have to do is find the best α , and then plug in to find β . Going forward the derivative will be one-dimensional. **Show the expression that will determine optimal α and β .**
- (h) Write a steepest descent algorithm for fitting the parameters in the gamma function to the length data. Just use a simple, fixed learning rate to start. You should stop your minimization when either the parameters change by only a small amount from one iteration to the next, or when the magnitude of the gradient is small (these are the same condition, so whichever you find easiest to code is fine). Since there may be noisy jumps in your minimization (because you aren't bracketing the minimum), make sure you keep track of the parameter set that gives you the smallest negative log likelihood throughout the run. If you want to check your steepest descent function before trying to fit the gamma, fit the exponential parameter above, since you know what the answer should be. If you get pretty close to the right μ , things are probably working.

- (i) Run your steepest descent algorithm for both a large (0.001) and a small (0.000001) learning rate and plot the cost as a function of gradient iteration. Does the negative log likelihood decrease? Do you see any noisy jumps? How similar are the parameters (α, β) you get in each run? Does the learning rate seem to matter much? (Hint: You may need to play around with your initial guess for α a bit to avoid numerical instability — if the cost is always `inf` or `nan`, try a different initial guess.)
- (j) Add the best fitting gamma distribution to your plot with the data and best exponential fit. Did you get a decent fit from your steepest descent routine? (If you did this homework problem for several different restriction enzymes in 1989, you could have published it — see Churchill et al. *Nucleic Acids Research*, **18**(3), pp. 589 (1989).)

What about a completely different kind of data where an exponential fit doesn't make any sense? For that, we'll switch to linguistics.

- (k) The file `wordlist.txt` has a list of about 20,000 English words (they come from something called the English Lexicon Project). Read these words into python and count how long (in letters) each one is. Make a plot of the probability that a word is a certain length — for this, you don't need any binning, because the longest English word in the data isn't nearly as long as the longest DNA fragment in the digested *E. coli* data. The easiest thing here is to just plot the lengths (1,2,3, etc.) against the counts you've computed. Divide each count by the total number of words in the data set (so the distribution sums to 1.0, as a proper probability distribution should).
- (l) Fit a maximum likelihood exponential distribution and gamma distribution to the data and plot the data and the two fits on the same plot. (This should be easy given all the code you've already written for the other parts of the problem). Which looks better? Is the exponential fit even close?

Basic steepest descent doesn't always work this well, but sometimes (as in deep learning) it's pretty much the only thing you can afford to do!